

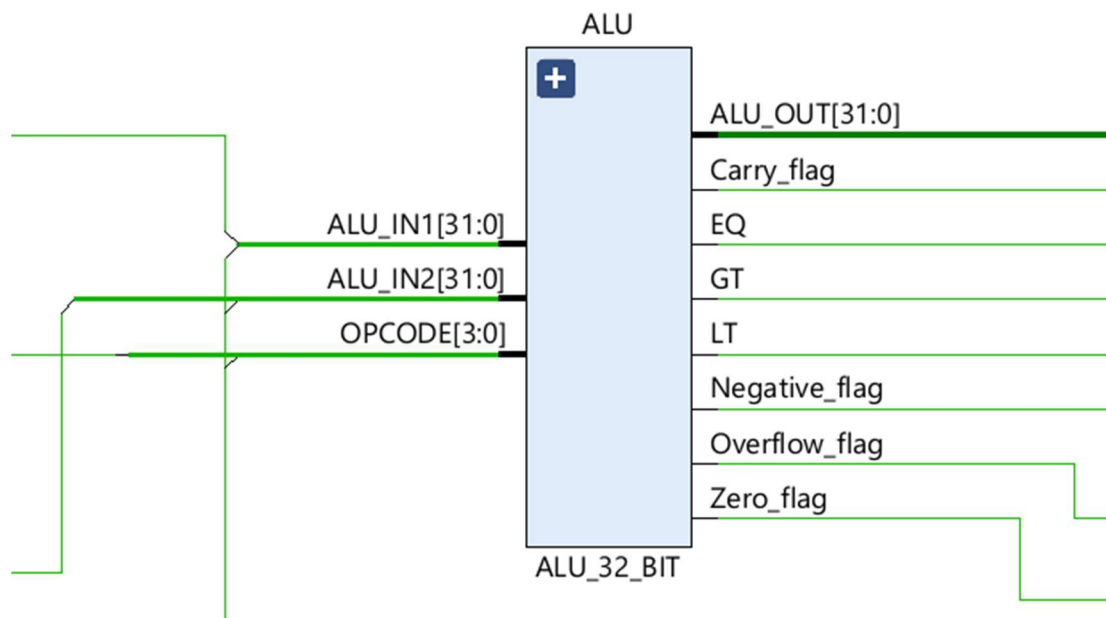
32-Bit ALU Detailed Description

Operation table, data path behavior, and status flag generation

ALU Purpose

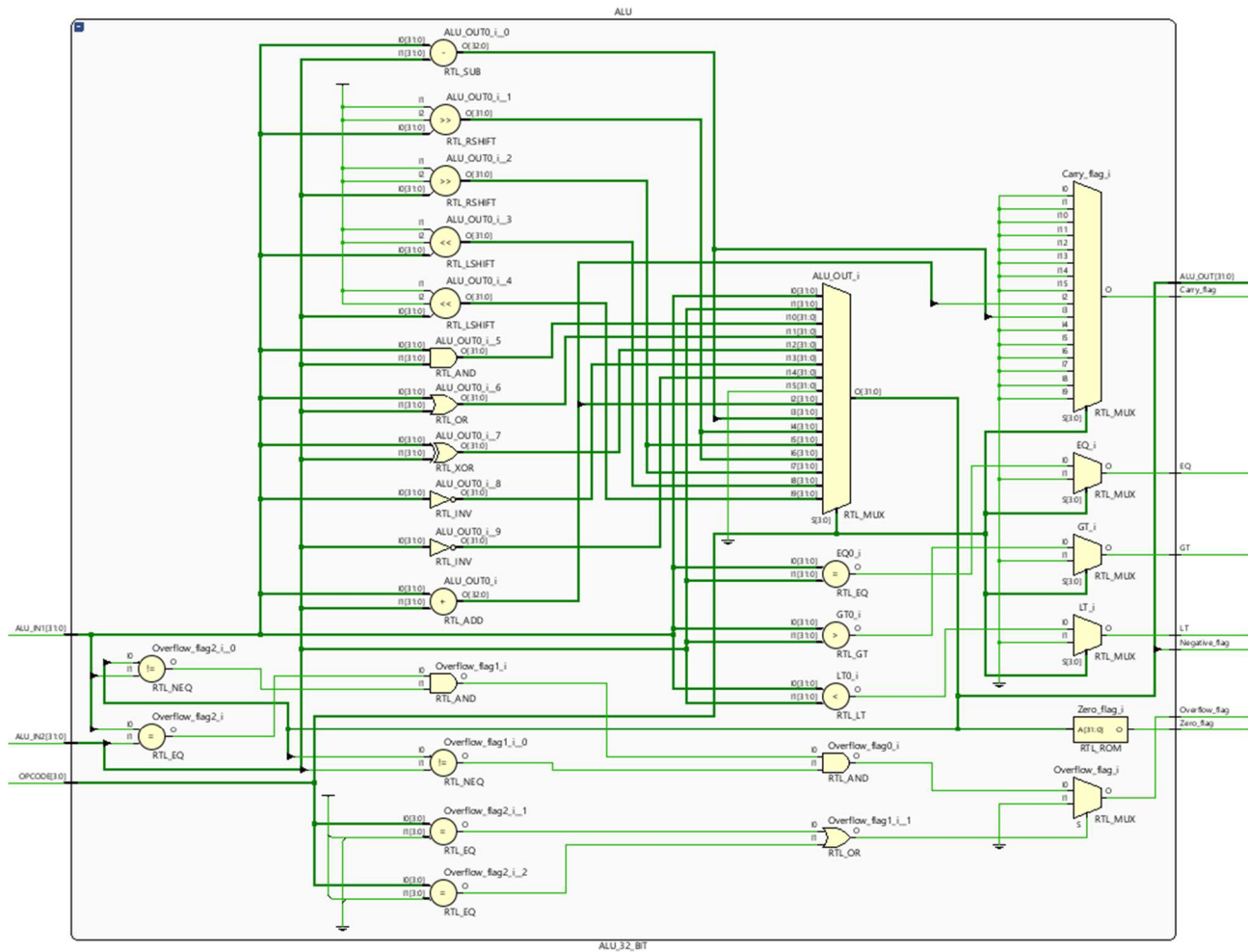
The ALU_32_BIT module is the execute block of the project. It takes two 32-bit operands and a 4-bit opcode, performs the requested operation, and generates the 32-bit result along with the carry, overflow, zero, negative, and compare status outputs. The ALU is purely combinational, so its outputs change immediately when the inputs or opcode change.

ALU Inputs and Outputs



- Inputs: ALU_IN1[31:0], ALU_IN2[31:0], and OPCODE[3:0].
- Outputs: ALU_OUT[31:0], Carry_flag, Overflow_flag, Zero_flag, Negative_flag, GT, LT, and EQ.
- The main result is ALU_OUT.
- The flag outputs describe the meaning of the result.

ALU Circuit Overview



Status Flag Behavior

- **Carry_flag:** Used for unsigned arithmetic carry or borrow information.
- **Overflow_flag:** Used for signed arithmetic overflow in add and subtract operations.
- **Zero_flag:** Set when the final ALU_OUT value is equal to zero.
- **Negative_flag:** Follows the most significant bit of ALU_OUT and indicates a negative signed result.
- **GT, LT, and EQ:** Compare outputs that describe the relationship between ALU_IN1 and ALU_IN2.

For arithmetic instructions, the flags give extra information about the result. For logical and shift instructions, the result still produces the zero and negative flags, while carry and overflow are generally not used in the same way as arithmetic instructions.

Opcode Mapping

OPCODE	Operation	Action
0000	Buffer A	Pass ALU_IN1 directly to output
0001	Buffer B	Pass ALU_IN2 directly to output
0010	ADD	$ALU_IN1 + ALU_IN2$
0011	SUB	$ALU_IN1 - ALU_IN2$
0100	SHR A	Logical right shift of ALU_IN1 by 1
0101	SHR B	Logical right shift of ALU_IN2 by 1
0110	Arithmetic SHR A	Arithmetic right shift of ALU_IN1 by 1
0111	Arithmetic SHR B	Arithmetic right shift of ALU_IN2 by 1
1000	SHL A	Logical left shift of ALU_IN1 by 1
1001	SHL B	Logical left shift of ALU_IN2 by 1
1010	Bitwise AND	$ALU_IN1 \text{ AND } ALU_IN2$
1011	Bitwise OR	$ALU_IN1 \text{ OR } ALU_IN2$
1100	Bitwise XOR	$ALU_IN1 \text{ XOR } ALU_IN2$
1101	Bitwise inversion A	NOT ALU_IN1
1110	Bitwise inversion B	NOT ALU_IN2
1111	Compare	Compare ALU_IN1 and ALU_IN2

Operation Details

The buffer opcodes are useful when the design needs to pass one operand through the ALU path without changing it. The add and subtract operations are the main arithmetic operations. The shift operations move the operand by one bit. Logical operations perform standard bitwise processing. The inversion operations produce the one's complement of the selected input.

The compare opcode does not produce a useful arithmetic result in ALU_OUT. Instead, it is used to generate the GT, LT, and EQ outputs so that the control logic can tell whether ALU_IN1 is greater than, less than, or equal to ALU_IN2.

Timing and Implementation Style

The ALU is implemented as combinational RTL. This means the calculation happens without waiting for a clock edge. The top-level pipeline registers hold the operands before and after the ALU stage, but the ALU itself is not clocked. This keeps the execute stage simple and fast.

How the ALU Fits the Whole Project

In the full pipelined system, the register file provides the operands, the control unit selects the opcode, the ALU computes the answer, and the writeback path stores the result back into the destination register. The ALU is therefore the core computation engine of the project.

Summary

This ALU supports a useful set of arithmetic, logical, shift, inversion, and compare functions. It is compact enough to be easy to verify, but complete enough to demonstrate real RTL design, status flag generation, and pipelined data flow.